

SECURITY REVIEW

Picon DLMM

A full-program security review of the on-chain Solana/Anchor program — authorization, value conservation, arithmetic, and cross-program invocation.

PROGRAM VERSION

1.1.2

COMMIT

694cdd6

DATE

29 August 2026

SCOPE

57 files · 9,432 lines

CRITICAL & HIGH

None found

MEDIUM · LOW · INFO

2 · 1 · 2

PREPARED BY

Claude Code

Anthropic

This review was performed by an AI system. It is a reasoned source-level analysis, not a substitute for an independent professional audit, and carries no warranty. Its scope and limitations are stated in §3 and §9.

1. Executive summary

Picon DLMM is a carefully and defensively written program. No critical or high-severity vulnerability was identified. Specifically, no path was found by which an unprivileged actor can steal principal, forge position ownership, bypass authorization, mint or duplicate a position NFT, corrupt bin accounting, or cause the program to sign a transfer it did not intend.

Five issues are reported: two medium-severity findings that are economic and operational rather than direct-theft, one low-severity latent robustness hazard, and two informational notes on the trust model. Neither medium finding is exploitable by an anonymous attacker against a correctly-operated deployment today; both concern value leakage or trusted-party capability.

The review also confirms the *absence* of a substantial list of bug classes that recur in automated market makers — fee-accrual ordering, iterator misalignment, reinitialization through `init_if_needed`, rounding that favors the user, unchecked arithmetic, and unvalidated account substitution. These are enumerated with their evidence in §8, because in a review the things that are correct are as much a result as the things that are not.

SEVERITY	FINDING	CLASS
MEDIUM	F-01 · Permissionless <code>sync_pool</code> suppresses the volatility fee	Economic — LP revenue leak
MEDIUM	F-02 · Allowlisted mint extensions grant issuers standing power over vaults	Trust — listing policy
LOW	F-03 · Mutate-before-validate relies on transaction atomicity	Robustness — latent
INFO	F-04 · Pool PDA does not canonicalize mint ordering	Operational
INFO	F-05 · Upgrade authority can unilaterally reset admin	Trust model

2. Severity rubric

Severity is assigned on realized impact combined with the preconditions an attacker must satisfy, not on theoretical worst case.

LEVEL	DEFINITION
CRITICAL	Direct, unprivileged theft of user funds, or total protocol compromise. No meaningful precondition.
HIGH	Loss of funds achievable under specific but attainable preconditions, or irreversible loss of protocol control.
MEDIUM	Systematic value leakage, denial of service against user funds, or a capability held by a trusted party that can cause loss if that party misbehaves or is compromised.
LOW	No present exploit path; a latent hazard, a defense-in-depth gap, or an invariant that is load-bearing but only implicitly enforced.
INFO	Design observation or trust-model property worth recording. No defect.

3. Scope and coverage

The review covered the entire `programs/picon-dlmm` crate: all 22 instruction handlers and their account-validation contexts, all 7 state modules, and the arithmetic, iterator and account-loading utilities. Coverage was complete for every module that carries value, validates an account, or performs arithmetic on user-controlled input.

MODULE GROUP	LINES	COVERAGE
Instruction handlers & contexts instructions/	1,932	Full — every handler, every <code>#[derive(Accounts)]</code> constraint
State machines state/	2,074	Full — bin, bin_array, pool, position, fee_state, dynamic_fee_state, admin_config
Account loading & validation bin_array_accounts/	747	Full — dense and sparse loaders, gap handling, PDA verification
Arithmetic bin_math, fee_math, u64x64, u128x128	1,143	Full — pricing, fills, fixed-point primitives, rounding directions
Deposit distribution distribution/	572	Full — weight math, k-derivation, per-bin dispatch
Token & CPI token, token_2022, transfer_fee, mint_extensions	539	Full — transfer construction, hook CPI, fee epochs, extension allowlist
Iterators & containers iterators, fixed_vec, remaining_accounts	990	Full — including the sole <code>unsafe</code> block in the crate
Range/index helpers, errors, events	~600	Reviewed through their call sites and invariant contracts

Out of scope: the off-chain quote library (`libs/picon-dlmm-quote`), the TypeScript SDK, the integration test suite, and the deployment/upgrade process. The `transfer-hook-adhoc` program was noted as a local-validator test fixture that is not deployed to devnet or mainnet, and was not reviewed as production code.

4. Threat model and trust assumptions

Findings are graded against the following actors. Making these explicit matters, because two findings are precisely about what a trusted actor is able to do.

ACTOR	TRUST	ASSUMED CAPABILITY
Anonymous trader / LP	Untrusted	Can call any permissionless instruction with arbitrary parameters and arbitrary accounts, in any order, and can compose several instructions in one transaction.
Position owner	Untrusted	Holds the position NFT; may attempt to act on positions or bin ranges that are not theirs.
<code>AdminConfig.authority</code>	Trusted	Creates pools, sets protocol share and dynamic-fee config, claims protocol fees, deletes empty bin arrays. Assumed to be a hardware-held key.
Program upgrade authority	Fully trusted	Can replace program code and reset the admin. This is the effective root of trust (see F-05).
Token mint issuer	Implicitly trusted	For any listed mint, retains whatever Token-2022 extension authorities that mint carries. This is the subject of F-02.

5. Review methodology and practice

This was a manual source-level review. It was structured rather than opportunistic: the program surface was mapped first, then read in dependency order, and each conclusion was traced to the specific code that enforces it rather than inferred from naming or comments.

5.1 Order of work

1. **Surface mapping.** Enumerated every source file with line counts, read the workspace and crate manifests to establish compiler-level guarantees, and read `lib.rs` to fix the complete set of external entry points (22 instructions).
2. **Entry points before internals.** Read each instruction's `#[derive(Accounts)]` context before its handler, since in Anchor the majority of authorization is declarative and lives in the context, not the body.
3. **Attacker-controlled inputs first.** Prioritized the paths where an untrusted caller supplies raw accounts — `remaining_accounts` grouping, the sparse bin-array loader used by swaps, and the transfer-hook CPI — because these are where account substitution and privilege forgery would appear.
4. **Value paths.** Traced every instruction that moves tokens (swap, deposit, withdraw, `claim_fee`, `claim_protocol_fee`) end to end, from parameter validation through state mutation to the CPI that actually transfers.
5. **Arithmetic last, but completely.** Read the fixed-point primitives, the pricing engine and the distribution math after the callers, so each function could be judged against the preconditions its actual call sites establish rather than in the abstract.

5.2 Techniques applied

- **Call-site closure on dangerous helpers.** Where a function is only safe under a precondition, every call site was enumerated by search rather than assumed. This is how `local_range_for` was cleared: it performs a signed subtraction cast to `usize`, so all three callers were checked to confirm each validates `contains_range` first.
- **Invariant tracing across modules.** Claims that make a bug impossible were followed to the code that enforces them. For example, the safety of admin bin-array deletion depends on a chain: `fee_owed` is stored in the position, deletion requires every bin empty, and bin emptiness is `total_shares == 0`, which cannot hold while any position holds shares — because deposit and withdraw mutate position shares and bin shares by the identical value.
- **Rounding-direction audit.** Every rounding site was checked for which party it favors, since a single inverted rounding direction in a hot path is a slow drain rather than an obvious bug.
- **Ordering analysis of state mutations.** Fee accrual, share mutation and reserve mutation were checked for relative order in each path, because these are correct only in one sequence.
- **Adversarial reading of iterator composition.** Zips and filters were checked for the order in which they compose, since filtering before zipping silently misaligns two sequences that are individually correct.
- **Compiler and language-level guarantees.** Confirmed `overflow-checks = true` in the release profile, so an unchecked arithmetic mistake aborts the transaction instead of wrapping silently; and reviewed the single `unsafe` block for memory safety independently of its callers.
- **Negative-space search.** Searched for absent controls, not only faulty ones — every `UncheckedAccount`, every `Signer`, and every `has_one` was enumerated and matched against where authorization was required.

5.3 What was deliberately not relied upon

- **Comments and naming.** Treated as intent, never as evidence. Every documented invariant was checked against the code that supposedly enforces it.
- **The existing test suite.** Used as a signal of author intent and as documentation of known trade-offs, but a passing test was never accepted as proof of a property. Tests confirm the cases they encode; a review must consider the cases they do not.
- **Prior review history.** Areas recorded as previously reviewed were re-derived independently rather than taken as settled.

6. Bug-class checklist

Classes actively hunted, and the outcome for each. "Clear" means a specific mechanism was identified that prevents the class, not merely that no instance was noticed.

CLASS	RESULT	MECHANISM RELIED ON
Missing signer / authorization	CLEAR	Every instruction carries an appropriate <code>Signer</code> ; admin paths use <code>has_one = authority</code>
Account substitution	CLEAR	Mints and vaults pinned by <code>address =</code> ; bin arrays validated on owner, discriminator, pool and index
PDA seed confusion	CLEAR	Pool signer seeds rebuilt from immutable stored fields; position PDA bound to its mint
Reinitialization via <code>init_if_needed</code>	CLEAR	Existing accounts take the <code>load()</code> -succeeds branch and are never re-initialized
Arithmetic overflow / wraparound	CLEAR	<code>overflow-checks = true</code> , pervasive <code>checked_*</code> , U256 intermediates
Rounding in the user's favor	CLEAR	Ceil on charges, floor on payouts, at every site
Fee-accrual ordering	CLEAR	<code>accrue_fee()</code> precedes every share mutation
Iterator misalignment	CLEAR	<code>filter</code> applied after <code>zip</code> , both derived from one range
Position NFT duplication	CLEAR	Mint authority revoked immediately after minting exactly 1
Unbounded / overcharging deposit	CLEAR	Double floor in k-derivation bounds the total by the declared cap
Hostile CPI via transfer hooks	CLEAR	Account metas resolved from the on-chain <code>ExtraAccountMetaList</code>
Unbounded input causing panic/DoS	CLEAR	Bin-array count and weight length bounded before any collection
Memory safety in <code>unsafe</code>	CLEAR	Single <code>FixedVec</code> block; write-after-assert, exact-count drop
Volatility-fee manipulation	F-01	—
Malicious token capability	F-02	—

7. Findings

F-01 · MEDIUM · ECONOMIC

Permissionless `sync_pool` suppresses the dynamic volatility fee

LOCATION	instructions/sync_pool.rs:52 · state/pool.rs:100–103 · state/dynamic_fee_state.rs:116–137 · utils/iterators.rs:30,51
IMPACT	LPs lose volatility compensation on large price moves. No principal at risk.
PRECONDITION	An empty bin region between the active bin and the liquidity being traded into.
ATTACKER	Any anonymous caller. No stake in the pool required.

`sync_pool` requires only a `Signer`. It moves `active_bin_id` and, through `sync_active_bin`, also rewrites the dynamic fee state's `reference_bin_id` to the same bin:

```
pub fn sync_active_bin(&mut self, bin_id: i32) {
    self.active_bin_id = bin_id;
    self.dynamic_fee_state.reference_bin_id = bin_id;
}
```

The dynamic fee is driven by `accumulate()`, which measures an *absolute* distance from the reference and is then squared when converted to a rate:

```
let index_delta = self.reference_bin_id.abs_diff(bin_id);
let accumulator = u64::from(self.volatility_reference)
    + u64::from(index_delta) * u64::from(MAX_BASIS_POINTS);
```

Separately, the swap iterators skip bins with no reserve on the outgoing side, so empty bins never reach `accumulate()`. The absolute-distance design is exactly what makes a jump across an empty gap chargeable: the landing bin's distance from the reference includes the whole gap, even though the intervening bins were never visited.

FAILURE SCENARIO

A pool's active bin sits in an empty region at distance D from the nearest liquidity. A swap would land on that liquidity and register `index_delta = D`, producing a volatility fee proportional to D^2 . Instead the trader submits, in a single transaction, a `sync_pool` call that walks `active_bin_id` and `reference_bin_id` across the gap to its far edge, followed by the swap. The swap now measures `index_delta  $\approx$  1`, reducing the volatility component of the fee by roughly a factor of D^2 . The sync costs only transaction fees and is available to anyone.

Waiting is not an equivalent substitute for the attacker, which is what makes this a real shortcut rather than an acceleration of normal behavior: `decay()` resets the reference to the *current* active bin, which remains on the far side of the gap.

REMEDIATION OPTIONS

- Do not reset `reference_bin_id` in `sync_active_bin`, leaving reference resets governed solely by `decay()`'s time filter. Smallest change; preserves sync's price-correction purpose while removing its fee effect.
- Prevent a sync and a swap on the same pool within one transaction.
- Accept and document, if the pricing effect is judged to be correct.

This may be intended behavior. Moving a stranded active bin to where liquidity actually sits is a legitimate price correction, and charging a volatility fee for correcting a stranded pointer would itself be wrong. The issue is narrower: the same primitive is available to a trader immediately before their own swap, in the same transaction. This warrants an explicit design decision and is reported for that reason rather than as an unambiguous defect. Sizing the real revenue impact requires modelling how often pools develop exploitable gaps in practice.

Allowlisted mint extensions grant token issuers standing power over pool vaults

LOCATION `utils/mint_extensions.rs:12–25` · `instructions/create_pool.rs:78–83`

IMPACT A listed token's issuer can drain or freeze that token's side of a pool.

PRECONDITION Admin has listed a mint carrying the relevant extension authority.

ATTACKER The token issuer, or whoever compromises the issuer's authority key.

`ALLOWED_MINT_EXTENSIONS` is a default-deny allowlist, which is the correct structure: an unrecognized or future extension is rejected rather than silently passing through, and `NonTransferable` is correctly excluded. However several *allowed* entries are not static properties of a token — they are capabilities its issuer retains and may exercise at any time after listing:

- **PermanentDelegate** — the delegate may transfer or burn tokens from *any* account of that mint, including the pool's vaults. This permits the complete removal of that token's reserves. It is the sharpest item in the list, and is rejected outright by a number of comparable protocols.
- **Pausable**, **DefaultAccountState** — allow the issuer to halt transfers or freeze accounts, locking LP funds for an indefinite period. Denial of service against withdrawals rather than theft.
- **TransferHook** — issuer-controlled CPI on every transfer. The program's own handling of this is sound (see §8), but the hook program's behavior remains the issuer's to change.
- **MintCloseAuthority** — additional issuer-controlled lifecycle surface.

FAILURE SCENARIO

A token behaves benignly and is vetted and listed. LPs deposit. At a later time of their choosing, the issuer exercises the permanent delegate and transfers the pool's entire vault balance of that mint to an address they control. The program has no available defense: the transfer is authorized by the token program itself, and no invariant in Picon DLMM is violated.

ASSESSMENT

The substantive mitigation is that `create_pool` is admin-gated, so an attacker cannot list a hostile mint on their own initiative. That is a real and significant control, and it is why this is rated Medium rather than High. The residual risk is that *vetting at creation time does not constrain future issuer behavior* — these are standing capabilities, not one-time properties observable at listing.

The recommendation is therefore to make this an explicit, written listing policy rather than an implicit one: either reject `PermanentDelegate` at the program level, or record that every listed issuer is fully trusted with that pool's reserves and vet accordingly — including who controls the extension authorities and whether they can be revoked.

Mutate-before-validate in `Bin::deposit` and `accrue_composition_fee`

LOCATION `state/bin.rs:168–176` · `state/bin.rs:262–268`

IMPACT None today. Latent double-counting hazard under future refactoring.

Both functions add to `amount_x`, `amount_y` and `total_shares` *before* validating the resulting liquidity against `max_bin_liquidity`, and return `None` with `self` already mutated:

```
self.amount_x = self.amount_x.checked_add(effective_amount_in_x)?;
self.amount_y = self.amount_y.checked_add(effective_amount_in_y)?;
self.total_shares = self.total_shares.checked_add(new_shares)?;

if bin_math::liquidity(self.amount_x, self.amount_y, self.price)? > max_bin_liquidity {
    return None; // self is already mutated
}
```

This is safe in the current program only because Solana discards all account state when an instruction returns an error, and every caller propagates the `None` as an error rather than handling it. The correctness of the code therefore rests on a property of the runtime, not on the function's own contract. Any future change that treats `None` as recoverable within the same instruction — retrying with a smaller amount, for instance — would double-count the partial mutation.

The codebase already recognizes this: two tests at `bin.rs:586–628` deliberately pin the current behavior so that a refactor cannot change it unnoticed. No code change is required. The recommendation is to state the dependency in the function's own documentation, so that it survives independently of the tests.

Pool PDA does not canonicalize mint ordering

LOCATION `instructions/create_pool.rs:47–59`

The pool PDA seeds use `token_mint_x` and `token_mint_y` in the order supplied by the caller, with only an inequality check and no canonical ordering such as a bitwise `x < y` constraint. Two distinct pools for the same token pair, with inverted price orientation, can therefore coexist — fragmenting liquidity and creating room for confusion about price direction in downstream integrations. Because `create_pool` is admin-gated this is an operational hazard rather than a vulnerability, and it is mentioned so that it is handled by listing procedure if not by code.

Upgrade authority can unilaterally reset admin

LOCATION `instructions/admin/reset_admin_authority.rs` · `instructions/admin/create_admin_config.rs`

Both instructions are gated on `program_data.upgrade_authority_address == Some(deployer.key())`, which allows the upgrade authority to take over `AdminConfig.authority` at will. This is correct and unavoidable — an upgrade authority could replace the program entirely — and the two-step admin handoff elsewhere is a genuinely useful control against key-transfer mistakes. It is recorded because it establishes that the *upgrade authority key, not the admin key, is the true root of trust* for the protocol. Securing it with a multisig is consequently the single highest-value operational control available, and it should be treated as such in deployment procedure.

8. Properties verified as sound

Each item below was confirmed against a specific enforcing mechanism. They are recorded because each would be a serious defect if it did not hold, and because a reader should be able to see what was actually checked rather than infer it from the absence of a finding.

8.1 Authorization and account validation

- **Signer coverage.** All 22 instructions were enumerated; each carries an appropriate `Signer`, and every admin path additionally binds it with `has_one = authority` against the singleton `AdminConfig` PDA.
- **Position ownership.** Uses the NFT pattern correctly — token account balance must equal 1, its owner must be the signer, and its mint is bound to `position.position_mint` in the handler. The position account itself is bound to its pool by `has_one = pool` on all three position-mutating instructions.
- **Unchecked accounts.** All five `UncheckedAccount` declarations were enumerated and each is validated — in the bin-array loader, by an `address = constraint`, or explicitly in the handler.
- **Bin-array validation.** `AccountLoader::try_from` enforces program ownership and discriminator; the loader then checks pool and index positionally against the expected range. Uninitialized gaps in the sparse loader must still match the canonical PDA, so a gap cannot be forged with an arbitrary account.
- **Duplicate-account resistance.** Passing the same bin array twice fails both on the runtime's shared-borrow semantics and on the loader's per-position index check.

8.2 Accounting and value conservation

- **Fee-accrual ordering.** Both deposit and withdraw call `accrue_fee()` before mutating shares, so newly added shares never harvest historical fees and exiting shares never forfeit earned ones.
- **Share synchronization.** Position shares and bin shares are always moved by the identical value in the same expression, preserving the invariant that a bin's `total_shares` equals the sum of position shares in it.
- **Fees survive bin-array deletion.** `fee_owed` is stored in the position, and deletion requires every bin empty — which cannot hold while any position retains shares. Rent additionally returns to the recorded payer, not to the admin.
- **Deposit charge is bounded by the declared cap.** The k -derivation floors twice, so $\sum[k \cdot w_i / 2^{128}] \leq \lfloor k \cdot W / 2^{128} \rfloor \leq \text{amount}$; the composition fee is subtracted from that budget rather than added on top.
- **Protocol fee is bounded.** Capped at 25% of fees actually collected, so it can never reach LP principal, and the claim fails closed if the vault is short.
- **Iterator alignment.** `claim_iter` and `withdraw_iter` apply `filter after zip`, and both sequences derive from the same range — avoiding the misalignment that would pay one bin's fees against another's state.

8.3 Arithmetic

- **Overflow discipline.** `overflow_checks = true` is set in the release profile, so an unchecked arithmetic error aborts the transaction rather than wrapping. Checked arithmetic is used pervasively regardless, and 256-bit intermediates are used where a product could exceed `u128`.
- **Rounding direction.** Uniformly favors the pool: ceil on amounts charged and on liquidity backing, floor on amounts paid out, on minted shares and on the protocol's cut. Dust accrues to the pool, never to the caller.
- **Exponentiation safety.** `exp_by_squaring` uses unchecked multiplication under a documented "base below 1.0" precondition. That precondition is structurally guaranteed by its only caller, which inverts the base before the call, so the panic path is unreachable.
- **Divide-by-zero.** Guarded explicitly in the fixed-point primitives, and a compile-time assertion enforces `MAX_FEE_RATE < FEE_PRECISION` so the gross-up denominator can never be zero.

8.4 Lifecycle, CPI and memory safety

- **init_if_needed safety.** The one use of this flag takes the `Load()` -succeeds branch for accounts that already exist and never re-initializes, closing the reinitialization class.
- **Position NFT integrity.** Mint authority is revoked immediately after minting exactly one token, so a position token can never be duplicated or re-minted.
- **Position closure.** Refuses to close while the position is non-empty, where emptiness accounts for both shares and unclaimed fees.
- **Transfer-hook CPI.** Account metas are resolved from the on-chain `ExtraAccountMetaList`, so caller-supplied account infos serve only as a lookup pool and cannot forge signer or writable privileges on the pool PDA.
- **Transfer fees.** `get_epoch_fee(clock.epoch)` correctly selects across the epoch boundary, and gross/net conversion is applied on the correct side of each leg in both swap directions.
- **Memory safety.** The crate's only `unsafe` is `FixedVec`'s `MaybeUninit` handling: `push` asserts capacity before writing, `Deref` exposes exactly `count` initialized elements, and `Drop` drops exactly `count`. All callers bound their input before collecting, so the panic-on-overflow path is unreachable from user input.
- **Bounds guarding.** Every call site of the signed-to-unsigned index conversion validates range containment first, preventing a wrap to a very large index.
- **Admin handoff.** Two-step nominate/accept prevents transferring authority to an uncontrolled key; the zero-pubkey initial `pending_authority` is not signable.

9. Limitations

This is a manual source-level review of a single commit. It does not constitute a guarantee of correctness, and the following are explicitly outside what it establishes:

- **No formal verification.** Invariants were established by reading and reasoning, not mechanically proven. Deep arithmetic properties over the full input domain — particularly conservation across long sequences of interleaved operations — are argued rather than exhaustively demonstrated.
- **No economic simulation.** F-01 in particular is characterized qualitatively. Its real revenue impact depends on how frequently pools develop exploitable gaps under live conditions, which requires modelling against production data to quantify.
- **Single-commit scope.** Findings apply to commit `694cdd6`. The off-chain quote library, the TypeScript SDK, and the deployment and upgrade process were not reviewed; a divergence between the on-chain math and the off-chain quote library would not have been detected here.
- **Composability with external protocols** — routers, aggregators and lending integrations — was not assessed.

Where a finding depends on a judgment about intent rather than on a demonstrable defect, that is stated in the finding itself rather than resolved silently in either direction.